실습 개요 소개

실습순서

1. 파이토치

1. 파이토치 기본
2. 회귀 Regression (ANN, DNN)
3. 분류 Classification (이진분류, 다중분류)

2. Tensorflow

1. 회귀 Regression (선형, 비선형, 다중)
2. 분류 Classification (이진분류, 다중분류)

PyTorch 00번 실습 - 파이토치 기본 - torch 모듈 사용

0. 파이토파이터치모듈 사용 -- anaconda. colab 은 이미 pytorch 모듈이 설치된 상태. [pip install torch]
1. tensor 자료형 : PyTorch의 가장 중요한 자료형 (파이토치 모델이 학습할 수 있는 데이터의 자료형)
   NumPy의 ndarray와 거의 비슷하지만 [GPU 사용 가능, 자동미분 가능] 이라는 차이가 있음
2. tensor 데이터 다루기, 자료형 변환

수행과제

교재(딥러닝 텐서플로 교과서)

- 1장 ~ 03장 까지 읽어보기 – 딥러닝기초와 개발환경, 머신러닝 핵심 알고리즘 (~ p.112)

교재(다양한 프로젝트로 끝장내는 파이토치 딥러닝)

- PART1. chapter 01 ~ 02 까지 읽고 예제 따라하기 (~ p.59)

NEXT...

인공신경망 ANN 입문 및 심층신경망(DNN)으로 머신러닝에서 수행했던 [회귀,분류]모델 만들어보기

PyTorch 01번 실습 - ANN 맛보기 - 시험점수 예측

인공 신경망으로 [공부시간]에 따른 [시험점수] 예측 모델 만들기

1개의 feature(공부시간)를 기반으로 시험점수를 예측하는 가장 단순한 인공신경망(ANN) 모델 만들기 - 단층 퍼셉트론을 이용한 선형회귀

[학습 데이터] : 공부시간 1~5시간에 따른 시험점수 5개 데이터 준비

[입력층]
1개의 특성 입력: 공부시간(x): 1, 2, 3, 4, 5

[출력층] 선형회귀 nn.Linear(1, 1)
입력값 1개를 받아 시험점수 1개 예측: 예측 점수 = 공부시간 × 가중치(w) + 편향(b)

처음에는 임의의 가중치와 편향으로 예측하므로 실제 시험점수와 오차 발생
[손실함수] MSELoss - 예측 오차 계산
예측한 시험점수와 실제 시험점수의 차이를 평균제곱오차(MSE)로 계산 - 얼마나 틀렸는지 평가

[역전파] backward() - 기울기 계산
오차를 줄이기 위해 가중치(w)와 편향(b)을 어느 방향으로 얼마나 수정해야 하는지 기울기 계산

[최적화] optimizer.step() - 가중치 갱신
경사하강법을 이용하여 오차가 줄어드는 방향으로 가중치와 편향 수정

[반복 학습] Epoch
예측 → 오차 계산 → 역전파 → 가중치 갱신 과정을 반복하여 공부시간과 시험점수의 관계를 학습하고 최적의 회귀 직선 찾기

PyTorch 01번 실습 - ANN 맛보기 - 시험점수 예측

scikit-learn의 LinearRegression은 fit()으로 회귀계수를 구하지만,

PyTorch는 **순전파 → 손실 계산 → 역전파 → 가중치 갱신**이라는 신경망 학습 과정을 **직접 구현**함.

단순한 선형회귀부터 신경망의 학습 원리를 이해하고, 이후 은닉층을 추가하여 복잡한 패턴을 학습하는 DNN(심층신경망)으로 확장하기 위한 기초 실습

NEXT...

인공신경망 ANN 입문 및 심층신경망(DNN)으로 머신러닝에서 수행했던 [회귀,분류]모델 만들어보기

PyTorch 02번 실습 - DNN Regression 자전거대여량 예측

딥러닝으로 [시간,기온,습도]에 따른 [자전거대여 수요]예측 모델만들기

3개의 feature들을 기반으로 가능한 다양한 특징을 만들어내자 - 자동 특성 추출

[입력층]
3개의 특성 입력: 시간, 기온 , 습도

[은닉층 #1] 파생 특성 layer (3, 256)
입력 3개받아 256개의 특성 만들기: 퇴근시간, 평일, 월요일, 화요일, ...온도 변화, 습도 변화, 비 영향 .....등

[은닉층 #2] 특성조합 layer (256,128)
256개의 특성을 조합하여 128개의 특성으로 만들기 : 평일 출근 수요, 주말 여가 수요, 날씨 민감도

[은닉층 #3] 특성조합 layer (128,64)
최종적으로 대여량을 결정하는 핵심 요인 64개의 특성 만들기: 출퇴근 수요 강도, 외출 친화도, 대여소 혼잡도 예상

[출력층]
64개의 특성을 기반으로 다중선형회귀를 수행한 결과 출력: 자전거대여 수요

머신러닝에서 데이터셋을 분석하여 파생변수 만들고 주성분 분석하여 줄이고 하던 개발자의 특성공학 Feature Engineering 작업을
딥러닝의 다층 퍼셉트론으로 알아서 **자동 특성 추출**

PyTorch 02번 실습 - DNN Regression 자전거대여량 예측

수행과제

교재(다양한 프로젝트로 끝장내는 파이토치 딥러닝)

- PART1. chapter 03 읽어보기. ANN의 개념과 한계를 XOR문제로 소개. (~ p.76)
- PART2. chapter 02 당뇨병 진행도 예측. 따라 구현해보기 (p.95 ~ p.104) ... chapter01은 굳이 안해봐도 됨.

코드를 직접 써보면서 딥러닝모델 코드 익히기(실무에서는 바이브코딩으로 하기에 숙달까지는 필요없음)

NEXT...

심층신경망(DNN)으로 지도학습 중 분류모델 만들어보기

PyTorch 03번 실습 - DNN Classification 이진 분류

딥러닝으로 [세포의 크기, 모양, 질감 등 30개 특성]을 분석하여 [유방암 악성/양성] 분류 모델 만들기

30개의 feature들을 기반으로 다양한 특징을 추출하고 조합하여 악성 종양과 양성 종양을 구분하는 패턴 학습하기 - DNN 이진분류(Binary Classification)

[학습 데이터] scikit-learn 유방암 데이터셋 - load_breast_cancer() 사용.
입력 데이터(X): 세포핵의 반지름, 질감, 둘레, 면적, 매끄러움 등 30개 특성
정답 데이터(y): 악성 종양(0), 양성 종양(1)
569개의 샘플을 학습하여 새로운 환자 데이터 분류

[입력층]
30개의 특성 입력: 세포핵의 크기, 형태, 질감, 대칭성, 오목함 등

[은닉층 #1] 파생 특성 layer (30, 64) + ReLU
입력 30개를 받아 64개의 조합 특성 만들기: 세포 크기 이상 정도, 형태 불규칙성, 표면 거칠기, 세포핵 변형 패턴 등 복합적인 특징 학습

[은닉층 #2] 특성조합 layer (64, 32) + ReLU
64개의 특성을 조합하여 32개의 핵심 특성 만들기: 크기와 형태의 복합 이상 패턴, 악성 의심 특징의 조합, 양성 종양의 특징 패턴 등

[출력층] 이진분류 layer (32, 1)
32개의 핵심 특성을 기반으로 최종 1개의 점수(로짓값) 계산
선형변환 y = wx + b의 결과인 로짓값을 Sigmoid 함수로 0~1 사이의 확률로 변환

PyTorch 03번 실습 - DNN Classification 이진 분류

[Sigmoid] 확률 변환 및 최종 분류
Sigmoid 출력 ≥ 0.5 → 양성 종양(1) 예측
Sigmoid 출력 < 0.5 → 악성 종양(0) 예측

[손실함수 및 학습] BCEWithLogitsLoss
예측 로짓값과 실제 정답의 차이를 손실로 계산하고 역전파를 통해 가중치와 편향을 반복적으로 수정하여 분류 성능 개선
Sigmoid + Binary Cross Entropy를 합쳐놓은 Loss 함수. (다중분류는 CrossEntropyLoss)

nn.BCELoss() : 모델의 출력층에 이미 sigmoid 활성함수가 있을때.
nn.BCEWithLogitsLoss() : 모델에 sigmoid 활성화 함수가 없을때.(권장)
권장하는 이유는 수치적 안정성(numerical stability) : Sigmoid와 BCE를 따로 계산하는 것보다, PyTorch가 둘을 결합해서 계산하는 BCEWithLogitsLoss()가 극단적으로 큰/작은 값에서도 더 안정적으로 계산

머신러닝에서 개발자가 데이터의 특징을 분석하여 파생변수를 만들고 분류에 유용한 특성을 선택하던 Feature Engineering 작업을 DNN의 은닉층에서 자동으로 학습
DNN 선형회귀가 연속적인 수치를 예측하는 모델이었다면, DNN Classification은 여러 은닉층에서 비선형 패턴을 학습하고 Sigmoid 함수로 확률을 계산하여 두 범주 중 하나로 분류하는 모델

NEXT...

머신러닝 수업에 했던 고객이탈가능성 DNN 분류 모델 만들기 추가 실습. 딥러닝은 연산이 많으니 GPU 사용해보기. NVIDIA CUDA 적용

PyTorch 04번 실습 - DNN Classification 고객 탈퇴 여부 예측 (GPU)

딥러닝으로 [가입기간, 로그인 빈도, 쿠폰 사용, 결제금액, 고객문의]에 따른 [서비스 탈퇴 여부] 예측 모델 만들기

5개의 고객 행동 feature들을 기반으로 다양한 특징을 추출하고 조합하여 서비스 탈퇴 가능성이 높은 고객 분류하기 - DNN 자동 특성 추출 및 이진분류

[학습 데이터] 고객 행동 데이터셋 - 머신러닝 실습에서 사용했던 고객 서비스 탈퇴 예측 데이터 활용

tenure_months: 서비스 가입 기간
weekday_login_7days: 최근 7일간 평일 로그인 횟수
coupon_to_purchase_min: 쿠폰 사용 후 구매까지 소요시간
monthly_spend: 월평균 결제금액
support_calls: 고객센터 문의 횟수
churn: 서비스 탈퇴 여부 (0: 유지, 1: 탈퇴)

[입력층]
5개의 고객 행동 특성 입력: 가입기간, 로그인 빈도, 쿠폰 사용 패턴, 결제금액, 고객문의 횟수

[은닉층 #1] 파생 특성 layer (5, 64) + ReLU
입력 5개를 받아 64개의 조합 특성 만들기: 장기 미활동 고객, 구매 전환 속도, 결제 활동성, 고객센터 의존도, 서비스 이용 감소 패턴 등

[은닉층 #2] 특성조합 layer (64, 32) + ReLU
64개의 특성을 조합하여 32개의 핵심 특성 만들기: 이탈 위험 행동 패턴, 충성고객 특징, 구매 활동 감소와 고객 불만의 복합 패턴 등

[출력층] 이진분류 layer (32, 1)
32개의 핵심 특성을 기반으로 최종 1개의 로짓값 계산. 로짓값에 Sigmoid 활성화 함수를 적용하여 고객의 서비스 탈퇴 확률(0~1) 계산

PyTorch 04번 실습 - DNN Classification 고객 탈퇴 여부 예측 (GPU)

[GPU 가속] CUDA를 활용한 딥러닝 학습
딥러닝은 수많은 가중치와 편향을 반복 계산하므로 병렬 연산에 특화된 GPU를 활용하여 학습 속도 향상
Google Colab에서 NVIDIA T4 GPU 런타임 선택 (VRAM 16GB, CUDA 코어 2,560개)
모델과 학습 데이터를 .to(device)로 GPU 메모리(VRAM)에 이동시켜 연산 수행

구글 코랩(Colab)에서 런타임 유형을 GPU로 변경했더라도(즉, GPU가 있는 학습환경이더라도) 코드에서 명시적으로 GPU를 사용하겠다고 지정하지 않았다면 딥러닝 연산은 CPU로 수행함. 모델 및 데이터를 GPU에서 구동되도록.. GPU의 vRAM으로 이동해서 수행하도록 해야 함.

파이토치(PyTorch)에서 .to("cuda")를 호출하면, 해당 데이터나 모델이 일반 시스템 메모리(RAM)에서 그래픽 카드의 전용 메모리인 VRAM(Video RAM)으로 복사되어 이동

GPU 사용 코드

```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

model = MyDeepLearningModel().to(device) # GPU로 이동
inputs = torch.randn(32, 3, 224, 224).to(device) # GPU로 이동
outputs = model(inputs) # GPU에서 고속 연산 처리
```

참고로 "CUDA"는 Nvidia의 고유 기술명칭이기에 학습환경에 사용되는 그래픽카드에 따라 고유기술명을 사용해야함.

PyTorch 04번 실습 - DNN Classification 고객 탈퇴 여부 예측 (GPU)

그래픽카드 제조사별 PyTorch 디바이스 명칭

그래픽카드 제조사 / 환경	가속 기술 이름	PyTorch에서 사용하는 디바이스 문자열
NVIDIA (지포스, 쿼드로 등)	CUDA	.to("cuda")
Apple (MacBook M1/M2/M3/M4 등)	MPS (Metal Performance Shaders)	.to("mps")
AMD (라데온, 인스팅트 등)	ROCm (Radeon Open Compute)	.to("cuda") 또는 .to("hip")
Intel (아크, 데이터센터 GPU 등)	OneAPI / XPU	.to("xpu")

[손실함수 및 학습] BCEWithLogitsLoss + 역전파
예측 로짓값과 실제 탈퇴 여부의 오차를 계산하고, GPU에서 역전파와 가중치 갱신을 반복하여 분류 성능 개선

기존 머신러닝에서 개발자가 고객 행동 데이터를 분석하여 파생변수를 만들고 특성을 선택하던 Feature Engineering 작업을 DNN의 은닉층에서 자동으로 학습
이전 유방암 이진분류 실습의 DNN 구조를 고객 행동 데이터에 적용하고, CPU 중심의 학습에서 CUDA GPU 병렬 연산을 활용하는 딥러닝 학습으로 확장하는 실습

NEXT...

DNN 다중 분류

PyTorch 05번 실습 - DNN Classification 붓꽃(Iris) 품종 다중 분류

딥러닝으로 [꽃받침 길이, 꽃받침 너비, 꽃잎 길이, 꽃잎 너비]에 따른 [붓꽃 3종류] 분류 모델 만들기

4개의 feature들을 기반으로 다양한 특징을 추출하고 조합하여 붓꽃의 품종을 구분하는 패턴 학습하기 - DNN 자동 특성 추출 및 다중분류

[학습 데이터] scikit-learn Iris 데이터셋 - load_iris() 사용 - 150개의 붓꽃 샘플과 4개의 특성
sepal length: 꽃받침 길이(cm), sepal width: 꽃받침 너비(cm)
petal length: 꽃잎 길이(cm), petal width: 꽃잎 너비(cm)
정답(y): 0 = setosa, 1 = versicolor, 2 = virginica

[입력층]
4개의 특성 입력: 꽃받침 길이, 꽃받침 너비, 꽃잎 길이, 꽃잎 너비

[은닉층 #1] 파생 특성 layer (4, 32) + ReLU
입력 4개를 받아 32개의 조합 특성 만들기: 꽃잎과 꽃받침의 크기 비율, 꽃잎의 길쭉한 정도, 꽃받침의 넓이 특성, 꽃잎과 꽃받침의 크기 차이 등

[은닉층 #2] 특성조합 layer (32, 16) + ReLU
32개의 특성을 조합하여 16개의 핵심 특성 만들기: 작은 꽃잎과 넓은 꽃받침 패턴, 길고 좁은 꽃잎 패턴, 품종별 꽃잎과 꽃받침의 복합적인 형태 차이 등

[출력층] 다중분류 layer (16, 3)
16개의 핵심 특성을 기반으로 3개 품종에 대한 로짓값(Logits) 출력
각 품종의 로짓값에 Softmax 활성화 함수를 적용하여 3개 품종에 속할 확률 계산 (전체 확률 합계 = 1)
출력 0: setosa 확률
출력 1: versicolor 확률
출력 2: virginica 확률

PyTorch 05번 실습 - DNN Classification 붓꽃(Iris) 품종 다중 분류

[Softmax] 확률 변환 및 최종 분류
3개 품종의 예측 확률 중 가장 높은 값을 가진 품종으로 최종 분류 - argmax()
예: setosa 0.05 / versicolor 0.85 / virginica 0.10 → versicolor(1) 예측

[손실함수 및 학습] CrossEntropyLoss + 역전파
3개 품종의 예측 로짓값과 실제 정답을 비교하여 손실을 계산하고 역전파를 통해 가중치와 편향을 반복적으로 수정하여 분류 성능 개선

이전 이진분류에서는 Sigmoid 함수로 하나의 확률을 계산하여 0과 1로 분류했다면,
다중분류에서는 Softmax 함수로 여러 클래스의 확률을 계산하고 가장 높은 확률의 클래스로 분류
DNN의 은닉층에서 꽃잎과 꽃받침의 복합적인 특징을 자동으로 학습하고, 출력층의 노드 수를 분류할 클래스 수(3개)에 맞춰 확장하여 여러 범주를 분류하는 모델

수행과제

교재(다양한 프로젝트로 끝장내는 파이토치 딥러닝)

- PART2. chapter 03 붓꽃(아이리스) 종류 분류 따라 구현해보기 (p.105 ~ p.112) ..책의 코드가 수업내용과 약간 다른 부분도 있으니 수행해보기
- **단, GPU에서 모델 학습 및 추론하도록**

코드를 직접 써보면서 딥러닝모델 코드 익히기(실무에서는 바이브코딩으로 하기에 숙달까지는 필요없음)

NEXT.. Tensorflow 모듈로 ANN, DNN 구현해보며 PyTorch와 비교해보기